Informe de seguridad de la SPA PharmaSoft — Sesión 11

Lenguaje de Programación II · UPeU · Ing. de Sistemas · Estudiante: Eliazar Cauna · Docente: Reyna Barreto Benjamin David · Rama: feature/seguridad-spa-pruebas-cauna

C1. Resumen de resultados

La matriz contiene 24 casos: los 16 base (L, S, I, G, M, E), 4 propios (P-01 a P-04, de los grupos Guards, Mi cuenta, Sesión y Accesibilidad) y los 4 del aviso de expiración (B-01 a B-04). **Estado al generar este documento: ningún caso manual se ha ejecutado todavía contra el backend real**, por lo que las columnas de ejecutadas, aprobadas y fallidas quedan sin cifras y deben completarse con la hoja «Resumen» de *docs/matriz-pruebas-s11.xlsx* cuando se hagan las pruebas.

Grupo	Casos planificados	Ejecutados	Aprobados	Fallidos
Login	4	[completar]	[completar]	[completar]
Sesión	5	[completar]	[completar]	[completar]
Interceptor	3	[completar]	[completar]	[completar]
Guards	3	[completar]	[completar]	[completar]
Menú y acciones	2	[completar]	[completar]	[completar]
Respuesta 403	1	[completar]	[completar]	[completar]
Mi cuenta	1	[completar]	[completar]	[completar]
Accesibilidad	1	[completar]	[completar]	[completar]
Aviso de expiración	4	[completar]	[completar]	[completar]
Total	24	[completar]	[completar]	[completar]

Comprobaciones técnicas sí realizadas (no sustituyen a la matriz): compilación de producción con *ng build* sin errores y 28 pruebas automáticas aprobadas (Vitest): servicio de sesión y cuenta regresiva, interceptor de token (API, login y dominio externo), componente del aviso, comparación de permisos y página «Mi cuenta».

C2. Fichas de hallazgos

No hay pruebas fallidas registradas porque la ejecución manual está pendiente; por eso no se incluyen fichas de fallos inventadas. Durante la revisión del código se corrigieron estos defectos y se dejan como hallazgos de revisión (no de la matriz):

Ref.	Problema	Sev.	Causa probable (archivo · método)	Corrección
H-1	El enrutador importa features/inicio/inicio, pero el ZIP no lo incluía: el build habría fallado.	Alta	app.routes.ts · ruta «inicio»	Se agregó un componente Inicio mínimo (conservar el propio si ya existe en el repositorio).
H-2	*autenticado* llamaba a token(), que puede cerrar la sesión; escribir una señal dentro de un computed lanza NG0600.	Media	auth-service.ts · autenticado	Ahora solo lee la señal de sesión; el vencimiento lo controla el temporizador único y token().
H-3	El login podía enviar un token anterior si quedaba una sesión vencida o ajena.	Baja	auth-interceptor.ts · authInterceptor	El interceptor no añade Authorization a /auth/login.
H-4	returnUrl con barra invertida («/\sitio») pasaba el filtro de «//».	Baja (hipótesis; el router de Angular no sale del origen)	login.ts · destino()	Se rechazan los valores con «\» además de «//».
H-5	Posible limitación: sessionStorage es por pestaña, así que cerrar sesión en una pestaña no cierra otra con sesión propia (caso S-04).	Por confirmar	auth-service.ts · cerrarSesion()	Hipótesis pendiente de verificar con S-04; aceptable por diseño en la guía, a documentar.

Evidencia de H-1 a H-4: ejecución de *ng build* y *ng test*. Cuando se ejecute la matriz, cada caso fallido recibirá una ficha con ID, descripción, severidad, evidencia (captura), causa probable y corrección.

C3. Preguntas de análisis

1. ¿Por qué el backend debe validar cada permiso aunque la SPA oculte lo que no se puede hacer?

Porque todo lo que corre en el navegador puede modificarse: el menú, la directiva **appSiPermiso* y los guards solo mejoran la experiencia. En el caso E-01 el vendedor recibe en su sesión un permiso adicional editando *sessionStorage*, el guard le deja abrir el formulario y envía la petición; la protección real es el 403 de PharmaBackend, que se verifica con el token firmado y no con lo que diga la interfaz. Sin esa validación cualquier usuario podría llamar a la API con una herramienta externa y saltarse la SPA. (El resultado observado de E-01 se registrará en la matriz al ejecutarla.)

2. Ventajas y riesgos de sessionStorage frente a localStorage y a una cookie HttpOnly; ¿cuál elegir para la botica?

sessionStorage se borra al cerrar el navegador y no se comparte entre pestañas, lo que conviene en computadoras compartidas; además la SPA descarta la sesión vencida o dañada al arrancar (S-02, S-03). localStorage persiste indefinidamente, por lo que el siguiente trabajador encontraría la sesión abierta. Ambos son legibles desde JavaScript, así que un XSS podría robar el token. Una cookie HttpOnly no es accesible desde JavaScript y resiste mejor al XSS, pero exige cambios en el backend (emisión de cookies, CORS con credenciales) y protección CSRF. Para la botica elegiría sessionStorage con tokens de vida corta, como está implementado, y consideraría la cookie HttpOnly como mejora futura si se pudiera modificar el backend.

3. ¿Por qué errorInterceptor cierra la sesión ante 401 pero no ante 403? ¿Qué pasaría al revés?

Un 401 significa que no hay identidad válida (token ausente, vencido o alterado), de modo que seguir con esa sesión no tiene sentido: se limpia y se envía al login recordando la página (I-03). Un 403 indica que el usuario sí está autenticado pero no tiene el permiso; la sesión es válida y el componente solo muestra el mensaje (E-01). Si se hiciera al revés, un permiso denegado expulsaría al usuario del sistema y un token vencido dejaría la sesión «viva» mostrando errores sin pedir que vuelva a ingresar. Además se excluye el login del cierre automático, porque allí el 401 significa «credenciales incorrectas».

4. ¿Qué es una redirección abierta y cómo la evita el returnUrl del login? (caso L-04)

Es una vulnerabilidad en la que la aplicación redirige a una dirección que controla el atacante, p. ej. un enlace /login?returnUrl=https://sitio-falso.com que, tras iniciar sesión, llevaría al usuario a una página que imita a la botica. El método *destino()* de login.ts solo acepta rutas que empiezan por «/», rechaza las que empiezan por «//» (URL relativa al protocolo) o contienen «\», y descarta /login; en cualquier otro caso navega a /inicio. Además *cerrarSesion()* aplica el mismo filtro al guardar el returnUrl. En L-04 se espera permanecer en localhost:4200/inicio con valores externos (resultado pendiente de registrar).